

Programming for Data Analytics II: Session 1

Managing Environments in Python with `uv`

Agenda

- Why Environment Management Matters?
- What is `uv` ?
- Why choose `uv` ?
- Installation (Mac & Windows)
- Creating & Activating Environments
- Managing Packages
- Best Practices

Why Environment Management Matters?

- **Isolation:** Prevent conflicts between project dependencies (e.g., Project A needs pandas 1.0, Project B needs pandas 2.0).
- **Reproducibility:** Ensure your code runs the same way on your machine, your colleague's machine, and the production server.
- **Cleanliness:** Keep your global Python installation clean and avoid "dependency hell".

What is `uv`?

- An extremely fast Python package installer and resolver.
- Written in **Rust**.
- Designed as a drop-in replacement for `pip` and `pip-tools`.
- Handles:
 - Python version management
 - Virtual environment creation
 - Package installation and resolution

Why `uv`?

1. **Speed:** It is significantly faster than `pip` and other tools due to its Rust implementation and efficient caching.
2. **Reliability:** Deterministic resolution ensures consistent environments.
3. **Simplicity:** Unified tool for managing Python versions, environments, and packages.
4. **Compatibility:** Compatible with standard Python tooling and workflows.

Prerequisite: Installing Python (Optional)

If you don't have Python installed yet (you should if you are using Anaconda, skip this step in that case):

- **macOS:**
 - Download from python.org or use Homebrew: `brew install python`
- **Windows:**
 - Download from python.org or use the Microsoft Store.
 - **Important:** Ensure you check "**Add Python to PATH**" during installation.
- **Verify:** Run `python --version` or `python3 --version`.

Installation: macOS

Open your terminal and run:

```
# Install using Homebrew (Recommended)
brew install uv

# Or using the standalone installer
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Verify installation:

```
uv --version
```

Installation: Windows

Open PowerShell and run:

```
# Using the standalone installer
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# Or using pip (if Python is already installed)
pip install uv
```

Verify installation:

```
uv --version
```


Initializing a Project

Navigate to your project folder and run:

```
# Initialize a new project (creates pyproject.toml)
uv init

# Or if you already have a pyproject.toml, just create the environment:
uv sync
```

`uv sync` creates a virtual environment (defaulting to `.venv`), installs dependencies from `pyproject.toml`, and generates a `uv.lock` file for reproducibility.

Activating the Environment: macOS / Linux

To start using the environment, you need to activate it.

macOS / Linux (zsh/bash):

```
source .venv/bin/activate
```

You should see `(.venv)` (or your project name) appear in your terminal prompt.

Activating the Environment: Windows

Windows (PowerShell):

```
.venv\Scripts\activate
```

Windows (Command Prompt):

```
.venv\Scripts\activate.bat
```

You should see `(.venv)` appear in your prompt.

Adding Packages

To install packages and add them to your `pyproject.toml` automatically:

```
# Add a single package
uv add pandas

# Add multiple packages
uv add numpy matplotlib scikit-learn
```

This updates `pyproject.toml` and `uv.lock`, ensuring everyone on the team has the same versions.

Managing Dependencies: `pyproject.toml`

We use `pyproject.toml` to define our project and dependencies.

```
[project]
name = "analytics"
dependencies = [
    "pandas>=2.2",
    "numpy",
]
```

This is the modern standard (PEP 518/621) for Python packaging, replacing `requirements.txt`.

Syncing Your Environment

To ensure your environment matches the lock file exactly:

```
# Sync environment with uv.lock  
uv sync
```

This installs missing packages, removes unused ones, and ensures your environment is perfectly reproducible.

Git & Version Control

What should you commit to GitHub?

- `pyproject.toml` :  **COMMIT**. Defines the project and abstract dependencies.
- `uv.lock` :  **COMMIT**. Locks exact versions for reproducibility.
- `.venv/` :  **IGNORE**. This is your local environment. It is specific to your machine and OS. Add `.venv` to your `.gitignore` file.

Deactivating & Removing

Deactivate:

To exit the virtual environment and return to your global Python:

```
deactivate
```

Remove:

To delete the environment, simply remove the `.venv` folder:

```
# macOS / Linux
```

```
rm -rf .venv
```

```
# Windows
```

```
Remove-Item -Recurse -Force .venv
```


Summary

- `uv` is a fast, modern tool for Python environment management.
- Always use **virtual environments** for your projects.
- **Workflow:**
 - i. `uv init` (Initialize)
 - ii. `uv add <package>` (Install & Track)
 - iii. `uv sync` (Sync & Lock)
 - iv. `source .venv/bin/activate` (Activate)
- Dependencies are tracked in `pyproject.toml` and locked in `uv.lock`.

Questions?

Time to set up your environments! Use AI to help you if needed, and don't hesitate to ask if you run into any issues.

For our next session we will need a fully working environment with the required packages installed.